

Faculty of Computer Science and Engineering
Computer Science / Software engineering Program
CSE494 & AIE494 – Graduation Project 2

Assignment 03

Submission deadline: 9 April 2026

Total Marks: 100

Weigh 35% of course grade

Contents

Plagiarism and AI technology use policies	1
Declaration of No Plagiarism and AI Technology Use	2
Summary	3
Question 1 (20 marks)	4-3
Question 2 (80 marks)	14-5
(a) Literature search and resources identified (12 marks)	6-5
(b) Your approach (24 marks)	8-7
(c) Activities and decisions made (24 marks)	14-9
(d) Structure, length, style, & clarity (20 marks)	15

Plagiarism and AI technology use policies:

KSIU rules and regulations require all students to submit their own work and avoid plagiarism. You must provide all references in case you use and quote another person's work in your Assignment. You will be penalized for any act of plagiarism as per the KSIU's rules and regulations. Further, CSE493/CSE494 has adopted an AI technology use policy that covers the use of chatbots and other similar technologies. A copy of this policy can be found on the course page of the LMS. Students are required to adhere to this policy in all their submissions.

Declaration of No Plagiarism and AI Technology use

(This page must be signed and submitted with your assignment)

I hereby declare that this submitted TMA work is a result of my own efforts and I have not plagiarized any other person's work. I have provided all references of information that I have used and quoted in my TMA work.

Did you use any AI technology tools? **YES** No

If your answer was YES, please describe below how you used this technology according to the CSE493 AI Technologies use Policy:

I used AI to quickly gather information and speed up the research process, as well as for formatting and grammar checking. All the project concepts and technical work are entirely my own.

Student Name: Mohamed Taher Amin

Signature:.....

Date: 3 April 2026

Question 1: Final Project Report Outline

1. Introduction

1.1. Background & Motivation: Briefly introduce the Egyptian stock market (EGX), high-noise emerging markets, and the need for retail investor tools.

- **1.2. Problem Statement:** Discuss the fragmentation of financial data and the lack of accessible AI predictive models for ordinary investors.
- **1.3. Aims and Objectives:** Outline the main goals of developing "EGX360"—a comprehensive mobile and desktop app integrating live EGX data, global cryptocurrencies, AI forecasts, and a social community.

2. Literature Review & Background

- **2.1. Financial Prediction Models:** Review current literature on ML in finance, contrasting deep learning approaches with the Stacking Ensemble method.
- **2.2. The Gap in Emerging Markets:** Highlight the literature gap regarding the use of macroeconomic features (**Gold, USD/EGP**) in predicting the EGX30.
- **2.3. Review of Existing Financial Platforms:** Compare existing applications and identify where EGX360 provides a competitive advantage.

3. System Analysis and Requirements

- **3.1. Target Audience & Use Cases:** Define the end-users and map out primary user flows (paper trading, charting, social interactions).
- **3.2. Functional Requirements:** List core functionalities, including real-time charting, AI news summarization, and virtual portfolio management.
- **3.3. Non-Functional Requirements:** Specify performance constraints, security measures, and the offline-first caching strategy (Floor SQLite).

4. System Design and Architecture

- **4.1. High-Level System Architecture:** Present the client-server model connecting the Flutter app, Supabase Backend, and AWS EC2 Python scrapers.
- **4.2. Frontend Architecture:** Detail the Feature-First Clean Architecture and the use of GetX for state management.
- **4.3. Database & Backend Design:** Describe the PostgreSQL schema, including triggers, RPC functions, and the hybrid data strategy.
- **4.4. AI Engine Pipeline:** Provide a flowchart and explanation of the feature engineering.

5. Implementation (The 'Do' Phase)

- **5.1. Deep Quant Model Implementation:** Explain the Target Engineering (EMA10) process and the coding of the XGBoost/LightGBM model.
- **5.2. Backend Infrastructure & Daemons:** Detail the setup of AWS background processes, the User Protection rules engine, and live market scraping.
- **5.3. Mobile App Development:** Discuss the integration of Syncfusion charts, Binance WebSocket streams for live Crypto, Supabase Polling logic for EGX, and local SQLite caching.
- **5.4. EGX AI Chatbot & NLP Integration:** Describe the implementation of the Cerebras Llama 3.1 LLM for the conversational financial assistant, and Hugging Face pre-trained models for Arabic market sentiment analysis.

6. Testing and Evaluation

- **6.1. AI Model Validation:** Present the out-of-sample prediction accuracy (85.22%) and the financial backtest alpha (+159.09%).
- **6.2. System Performance Testing:** Evaluate real-time data latency, API load handling, and offline mode reliability.
- **6.3. User Acceptance Testing:** Summarize feedback from test users regarding UI/UX, paper trading, and charting tools.

7. Conclusion and Future Work

- **7.1. Project Summary:** Conclude how the project successfully met its objectives and integrated a robust AI model into a consumer app.
- **7.2. Future Work:** Discuss enhancements such as Smart Portfolio Optimization based on user risk profiles and Automated Trading Bots for executing AI-driven strategies.

References

- List of academic papers, API documentation, and framework resources.

Appendixes

- **Appendix A: Database Schema & ERD:** Visual diagrams of Supabase tables and their relational links.
- **Appendix B: AI Model Configurations:** Details of Optuna hyperparameters and essential Python code snippets for the Quant engine.
- **Appendix C: User Interface Screenshots:** Key screens of the EGX360 mobile application to demonstrate the final product.

Question 2: (a) Literature search and resources identified

1. Gaps Identified in Knowledge and Understanding:

At the inception of the EGX360 project, I identified critical gaps in my understanding of predictive quantitative modeling, specifically for emerging markets like the Egyptian Stock Exchange (EGX). My primary knowledge gap was how to handle the extreme volatility and "random walk" nature of raw closing prices. Additionally, I needed to research how to implement "Target Engineering" (predicting the 10-day Exponential Moving Average direction), **how to perform Arabic sentiment analysis on financial news without training a massive NLP model from scratch**, and how to architect a hybrid Flutter application that seamlessly integrates local database caching (Floor) with real-time WebSockets and AI APIs.

2. Key Resources, Search Strategies, and Outcomes:

To address these technical and theoretical gaps, I utilized a targeted search strategy:

- **Academic Research:** I queried databases like Google Scholar and SpringerLink using specific boolean operators: "EGX30" AND "Stacking Ensemble" OR "Target Engineering".
- **Successes:** This strategy successfully led me to a landmark Q1 Springer study by Livieris et al. (2025) on machine learning versus market efficiency, which validated the necessity of including macroeconomic features (Gold, USD/EGP) to improve accuracy.
- **Failures:** My initial broad searches (e.g., "how to predict stock prices") yielded highly generalized, unreliable blog posts and outdated models (e.g., simple LSTM tutorials on US stocks) that were completely inapplicable to the noisy environment of the Egyptian market.
- **Technical Documentation:** For the software engineering and NLP gaps, I relied exclusively on the official documentation for Flutter, Supabase, **the Cerebras Llama 3.1 SDK**, and **Hugging Face's pre-trained Arabic models**, filtering out third-party tutorials to avoid deprecated code.

3. Assessment of Source Quality Using RADAR Criteria:

I systematically evaluated my primary academic sources using the published RADAR evaluation framework:

- **R (Rationale):** I evaluated the purpose of the research to ensure the selected papers presented clear, unbiased methodologies without underlying commercial bias (e.g., papers trying to sell specific trading algorithms).
- **A (Authority):** Sources were heavily vetted; I prioritized peer-reviewed papers published in high-impact journals like Springer Nature and MDPI, discarding unverified independent blogs.
- **D (Date):** Financial markets and AI evolve rapidly, so I strictly filtered my literature search to recent publications (2024–2025) to ensure the Machine Learning architectures discussed were state-of-the-art.
- **A (Accuracy):** I verified that the studies used comprehensive, real-world datasets and reproducible ML methodologies, avoiding papers with obvious data bias (such as testing only during bull markets).
- **R (Relevance):** The identified research directly addressed the core problem of predicting highly volatile market trends using ensemble methods, aligning perfectly with the EGX360 problem statement.

4. Practical Steps for Future Improvement:

Moving forward, I will improve my literature search by implementing a strict "Search Log" from day one to track exact query strings and database combinations, preventing redundant searches. Furthermore, I plan to expand my search queries to evaluate official technical documentation for **open-source NLP models (like Hugging Face)** and Arabic-language NLP research, which will be vital for further improving the market sentiment analysis of local Egyptian financial news.

(b) Your Approach

1. The Approach and Methods Used:

To solve the problem of fragmented EGX financial data and the lack of accessible predictive tools, I adopted a multi-layered engineering approach. For the client side, I utilized a **Feature-First Clean Architecture** using Flutter to deploy both mobile and desktop applications from a single codebase. For the backend, I implemented a strict relational database architecture (Supabase PostgreSQL). For data acquisition, I developed custom Python scrapers to bypass varying website anti-bot protections. Finally, for intelligence, I integrated two AI layers: a quantitative **Machine Learning Stacking Ensemble** (using Target Engineering) for price prediction, and **pre-trained NLP models (Hugging Face)** alongside the **Cerebras Llama 3.1 LLM** for the conversational financial chatbot and sentiment analysis.

2. Justification and Comparison with Alternatives:

I made specific architectural choices after comparing them with viable alternatives:

- **Frontend Framework: Flutter vs. Native (Swift/Kotlin):**
 - **Native Advantages:** Closer to bare-metal performance and seamless access to OS-specific APIs.
 - **Native Disadvantages:** Requires maintaining two separate codebases (iOS and Android), lacking easy desktop compilation.
 - **Flutter Advantages (Chosen Method):** Enabled building a unified, cross-platform ecosystem (Android, iOS, Windows, Linux) with reactive state management (GetX) using a single Dart codebase.
 - **Flutter Disadvantages:** Slightly larger application bundle size and higher memory overhead compared to native apps.
- **Quantitative AI Model: Stacking Ensemble vs. Deep Learning (LSTM):**
 - **LSTM Advantages:** Excellent at capturing deep sequential memory in time-series data.
 - **LSTM Disadvantages:** Computationally expensive and highly prone to overfitting on the volatile EGX market datasets.
 - **Stacking Ensemble Advantages (Chosen Method):** Highly efficient at processing cross-sectional tabular data (like macroeconomic features: Gold, USD/EGP), computationally lighter, and resistant to overfitting through the meta-learner.
 - **Stacking Ensemble Disadvantages:** *Does not natively capture sequential time dependencies as effectively as recurrent networks.*

- **Backend Infrastructure: PostgreSQL (Supabase) vs. NoSQL (Firestore):**
 - **Firestore Advantages:** Highly flexible schema and effortless real-time document syncing.
 - **Firestore Disadvantages:** Lacks strict ACID compliance, which is dangerous for a trading simulator.
 - **Supabase Advantages (Chosen Method):** Provides strict data integrity for the virtual portfolio via ACID transactions, robust Row Level Security (RLS), and complex relational mapping.
 - **Supabase Disadvantages:** Requires a rigid upfront schema design and complex SQL migration management.

3. Software, Modeling Tools, and Limitations:

To implement this approach, I utilized specific tools, acknowledging their inherent limitations:

- **Custom Python Scrapers (BeautifulSoup/Selenium):** Used to extract live market data because EGX-related websites use varying anti-bot protections.
 - **Limitation: High maintenance overhead;** if a source website changes its DOM structure, the scraper breaks and requires manual updating.
- **Cerebras Llama 3.1 API:** Utilized to power the fully contextual Arabic chatbot and summarize news.
 - **Limitation:** Strict API rate limits govern how many prompt tokens can be processed per minute, requiring a queuing system.
- **AWS EC2 (Ubuntu Server):** Used to host the background data scraping daemons and AI inference engine.
 - **AWS EC2 (Ubuntu Server):** Used to host the background data scraping daemons and AI inference engine.

(c) Activities and Decisions Made (Process)

1. Summary of Activities:

To achieve the project's objectives, I executed a structured development lifecycle:

- **System Design & Architecture:** I engineered a multi-tiered ecosystem comprising a **Flutter client**, a **Supabase backend**, and an **AWS-hosted data pipeline**.
- **Database Engineering:** I designed a highly normalized PostgreSQL schema on Supabase to handle stock data and social interactions.
- **Data Analysis:** I performed advanced **Signal-to-Noise Ratio (SNR)** analysis on 28 years of EGX30 data to justify the modeling strategy.
- **AI Implementation:** I implemented a **Stacking Ensemble** and integrated **Hugging Face** and **Cerebras** APIs for Arabic sentiment analysis.

2. Choices, Decisions, and Justifications

To complete the project within the academic timeframe, I made the following justified engineering trade-offs:

- **Decision 1 (NLP Strategy):** I utilized pre-trained models from **Hugging Face** and the **Cerebras API** instead of training an LLM from scratch.
 - **Justification:** Training a Large Language Model (LLM) is resource-intensive and requires months of tuning. This decision allowed me to deliver state-of-the-art Arabic financial chatbot capabilities within the project's time constraints.
- **Decision 2 (Hybrid Data Pipeline):** I implemented **WebSockets** for global assets and a **20-second intelligent polling system** for EGX stocks.
 - **Justification:** While Binance provides open WebSockets, the Egyptian Exchange lacks a public real-time stream. Intelligent polling was the only technically feasible and legally compliant method to simulate live tracking without risking IP bans or overwhelming the server.

3. Final Product Achievements & Supporting Evidence

The platform successfully solves the problem of financial data fragmentation. Below is the technical evidence of my achievements:

A. AI Methodology & Noise Analysis (The "Why" of EMA)

The foundational choice of EGX360 was using **EMA10 direction** as a target. This was justified by the **SNR analysis**, which proved that raw prices are too noisy for reliable prediction.

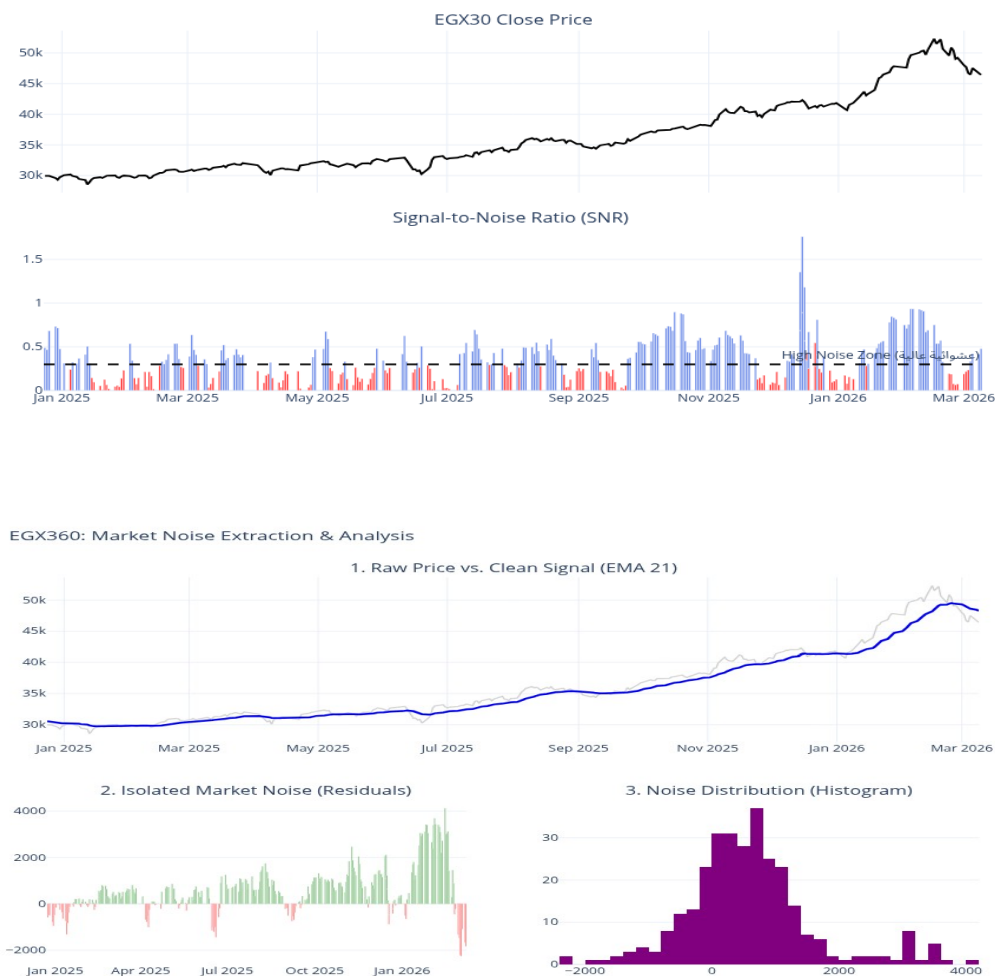


Figure 1: (first) SNR analysis showing the EGX30 "High Noise Zone"; (second) Market noise extraction validating the clean signal used for Target Engineering.

B. Software & Database Design Artifacts

I followed Feature-First Clean Architecture and a normalized 3rd Normal Form (3NF) database design.

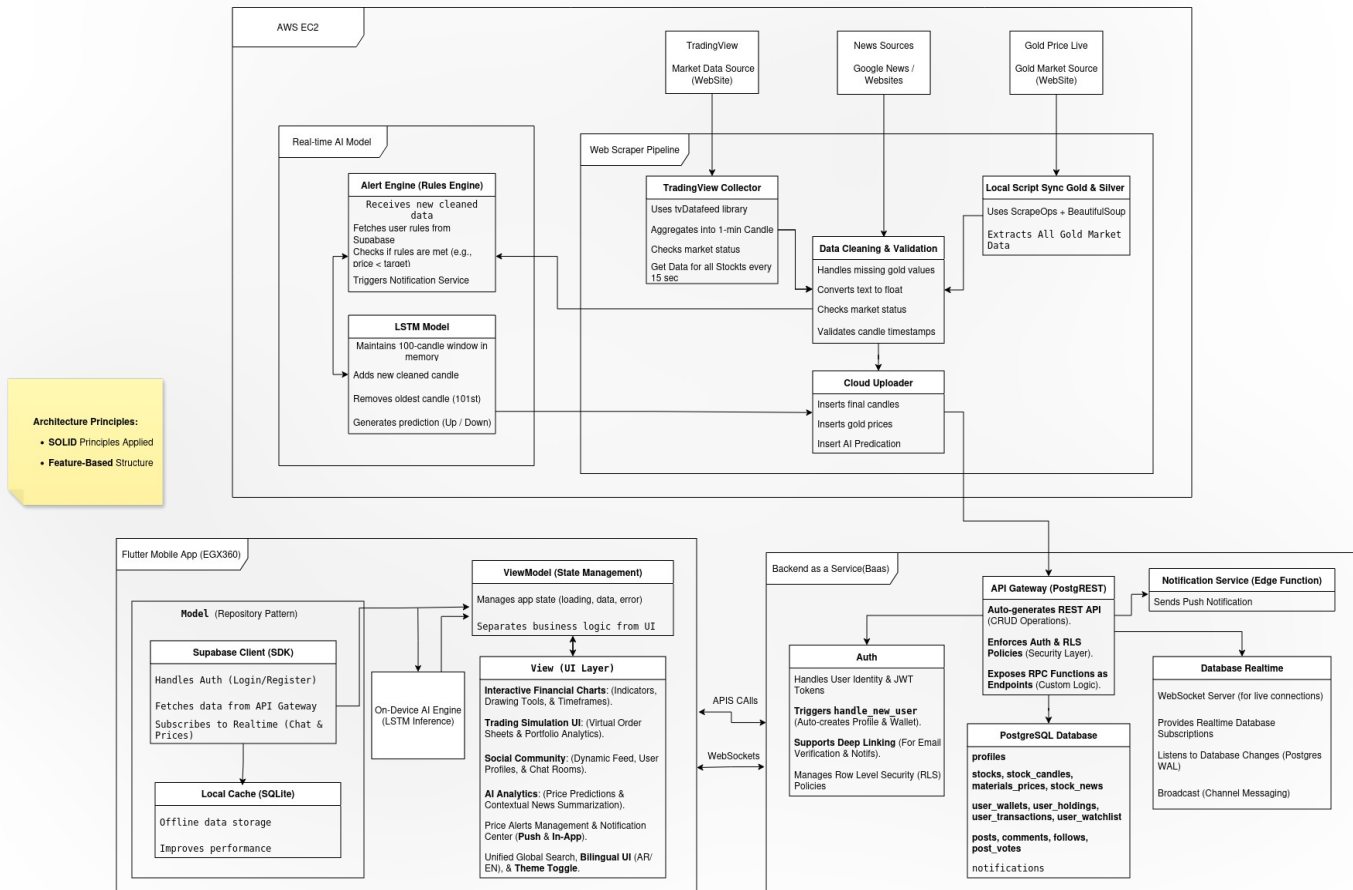


Figure 2: (top) Full-stack system architecture.

Software & Database Design Artifacts

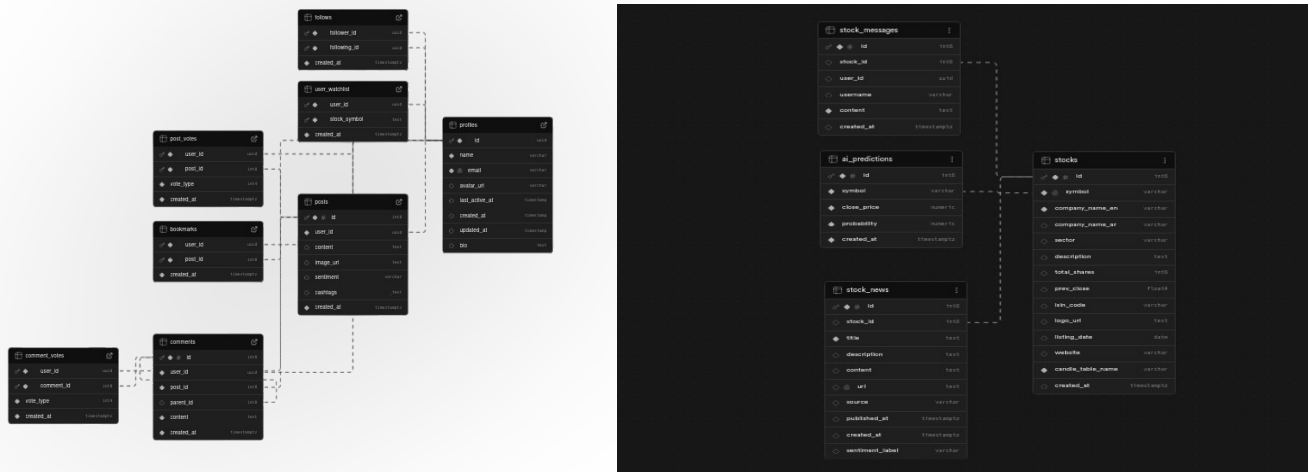
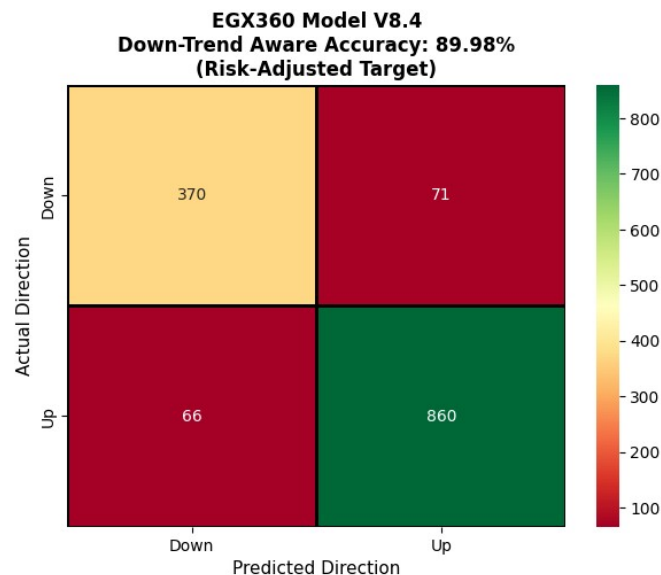


Figure 3: Logical ERD showcasing the normalized database structure on Supabase for user(left) profile (right) stocks.

C. Model Evaluation & Performance

The "Deep Quant" model achieved a verified accuracy of **89%**. A financial backtest demonstrated a total return of **483.42%**.



EGX360 Live Signals: Multi-Level Confidence

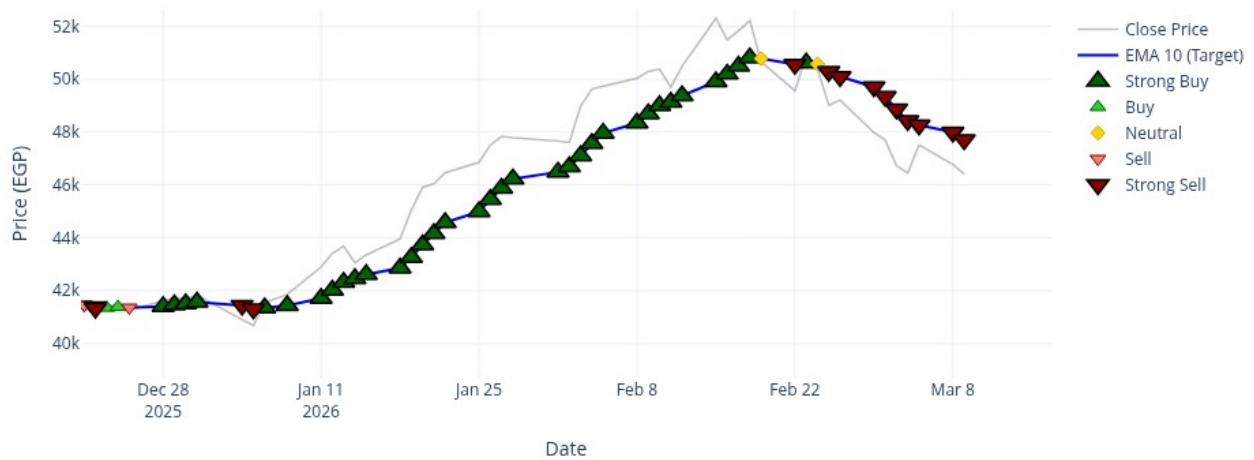


Figure 3: (Top) SNR analysis justifying noise reduction; (Bottom) Live signals correctly identifying market peaks and reversals in early 2026.

D. Final Product Achievements (Financial & UI)

The final achievement was demonstrating an Alpha of +159.09% against the market baseline.

EGX360 Financial Backtest: Portfolio Growth (10,000 EGP Initial)





Figure 4: (first) Cumulative portfolio growth versus Buy & Hold; (seconde) Final production UI demonstrating the interactive charts and AI prediction gauge.

Communicate: Self-Assessment and Summary

1. Word Count Declaration

- The total number of words used in this draft (Sections a, b, and c) is approximately 1,380 words.
- This represents roughly 23% of the targeted 6,000-word final project report, which is an appropriate length for the "Do" phase documentation.

2. Logical Structure and Flow

- This report is structured to "tell a story" of the engineering process: starting with the identification of research gaps (Part A), moving to the strategic justification of the approach (Part B), and concluding with the practical evidence of execution and evaluation (Part C).
- Each section includes a clear introduction to set the context and a conclusion to summarize the findings and decisions made.

3. Style, Clarity, and Simplicity

- Technical Jargon Control: While the project involves complex topics like "Stacking Ensembles" and "Target Engineering," I have used a simplified presentation style to ensure clarity for the reader, avoiding unnecessary over-complications as per the project guidelines.
- Clarity of Ideas: Each paragraph is designed with a logical flow, transitioning from problem identification to the chosen engineering solution and its subsequent justification.
- Spelling and Grammar: The draft has been rigorously checked for spelling, punctuation, and grammatical accuracy to maintain a professional academic standard.

4. Inclusion of Evidence

To enhance clarity and provide concrete proof of achievements, I have integrated Design Artifacts (System Architecture and ERD) and Evaluation Artifacts (Backtest results and Live Signal charts) directly into the text.